

End-of-Module (EoM) Project Brief (Version 1)

By Jonathan Loo

Coursework Title: Fully Embedded 2D-LiDAR based Autonomous Mobile Robot (AMR)

Coursework Type: Group project

Marks contribution: 21% of total module marks

1. Project Overview

This EoM project serves as the capstone for the Microprocessor Systems Design module. You will conceive, design, implement, and operate a fully embedded 2D-LiDAR-based AMR that functions under realistic, real-time, and resource-constrained conditions. The project reflects professional embedded-robotics engineering design and is inspired by the industry-grade AMR illustrated in Figure 1.

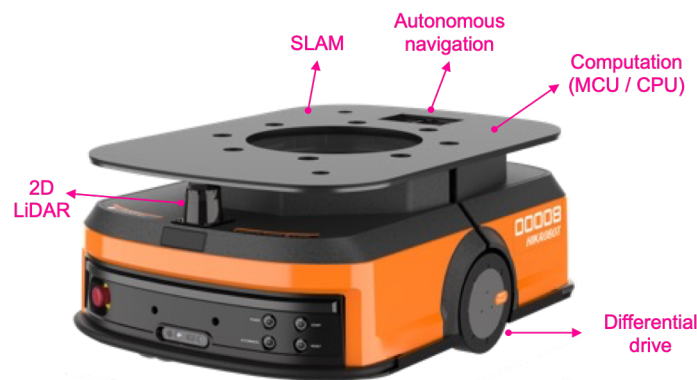


Figure 1. Industry-grade AMR

Your team will develop a complete **Sense → Think → Act** pipeline that executes entirely on the NUCLEO-F446RE (STM32F446RE microcontroller). All perception, localisation, mapping, planning, and control algorithms must run on-board; no external PC-based runtime computing is permitted.

Your AMR must demonstrate the following minimum behaviours:

- Autonomous exploration of an unknown maze
- 2D LiDAR-based environment perception
- Embedded localisation and mapping using odometry and scan data
- Goal-directed navigation from Start → Exit and Return-to-Start using the generated map
- Real-time closed-loop motor control under hardware and timing constraints

These capabilities must be implemented in a manner that reflects disciplined engineering judgement, hardware-software coherence, and real-time integration.

You will approach this project challenge using our GenAI-CDIO Authentic Assessment framework ([see Assessment Guidance on QMplus](#)), which emphasises:

- The engineering process as the primary focus
- The final system product as evidence of that process
- Performance benchmarking as objective validation

The assessment evaluates your ability to:

- Interpret complex engineering challenges
- Develop feasible and realistic solutions within the project timeframe
- Design a coherent and technically sound embedded system architecture

- Justify design trade-offs and limitations under practical constraints
- Validate performance using structured benchmarking
- Demonstrate engineering judgement through transparent, evidence-based reasoning

This project facilitates scaffolded and experiential learning. You will encounter intentional knowledge gaps designed to prompt investigation, critical thinking, and the exercise of engineering judgement. You are expected to apply structured reasoning across the full CDIO lifecycle: **Conceive → Design → Implement → Operate**.

You are encouraged to use GenAI tools to:

- Explore unfamiliar technical concepts (e.g., PID control, ICP scan matching, occupancy-grid mapping, A* search)
- Compare and evaluate alternative design approaches
- Support firmware development and debugging
- Reasoning about systems architecture and validation strategies

However, you remain fully responsible for:

- Verifying all AI-assisted outputs
- Ensuring technical accuracy and engineering validity
- Making and clearly justifying your own design decisions

This project is designed to develop your Engineering Competence (EC) through an experiential, end-to-end engineering workflow structured across the full CDIO lifecycle. Through systematic problem framing, design justification, implementation under constraints, and performance validation, you will progressively strengthen your ability to apply disciplined engineering reasoning in authentic contexts.

By the end of the module, you will have realised an integrated autonomous system, validated its behaviour through structured testing and benchmarking, and demonstrated your capacity to analyse, justify, and refine engineering decisions under realistic technical and resource constraints.

2. Standardised Hardware Kit

Each team receives a standardised hardware kit as shown in Figure 1 to ensure benchmarking fairness:

- STM32 NUCLEO-F446RE
- 2D LiDAR scanner (UART interface)
- MC520 Geared DC Motors with AT8236 motor driver (Timer/PWM interface)
- HC-04 Bluetooth module (UART interface)
- SSD1306 OLED display with 4 push buttons (I2C and GPIO interface)
- 3-gang Potentiometers (ADC interface)
- Logic analyser (signal/data analysis)
- 12V battery pack and charging adaptor

All teams will operate under identical hardware conditions to ensure fairness in quantitative benchmarking.

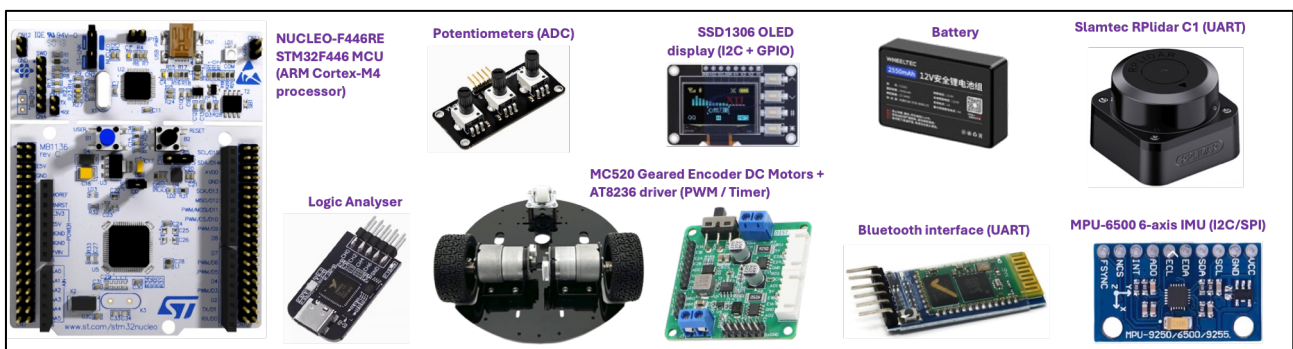


Figure 1. Standardised Hardware Kit

3. Core Technical Requirements

Your system must implement a complete embedded autonomy pipeline structured around: **Sense → Think → Act**

All components must execute entirely on the STM32 microcontroller and operate under memory and real-time constraints.

Your system must satisfy the following minimum functional requirements.

- a) Sense – Embedded Perception
 - Acquire 2D LiDAR scan data via an appropriate UART interface
 - Acquire wheel-encoder data using interrupt-based timer capture
 - Implement basic filtering or pre-processing to manage sensor noise
 - Sensing must be reliable, synchronised, and integrated into the real-time control pipeline
- b) Think – Embedded Cognition
 - Localisation and Mapping
 - Implement wheel-encoder-based odometry using differential-drive kinematics to estimate robot pose (x, y, θ) in real time
 - Implement a 2D Occupancy Grid Map (OGM) as the primary environment representation
 - Implement lightweight scan matching (e.g., ICP or equivalent) to refine pose estimation and mitigate cumulative odometry drift
 - Update the OGM incrementally using LiDAR data
 - Manage map resolution, memory allocation, and boundary conditions within embedded resource constraints
 - Navigation and Planning
 - Implement autonomous exploration (e.g., frontier exploration) based on the global path planning (e.g., A* or equivalent) operating on the OGM representation
 - Implement local obstacle avoidance (e.g., DWA)
 - Demonstrate successful **Start → Exit** and **Exit → Start** navigation.
- c) Act – Embedded Control
 - Implement closed-loop motor control
 - Implement PID-based velocity or position control
 - Justify tuning methodology
 - Maintain stability and responsiveness under real-time constraints.
- d) User Interface – Embedded Interaction
 - Implement an embedded user interface to support system configuration, monitoring, and safe operation.
 - Use the OLED display to present key system states and real-time status information without blocking real-time tasks.
 - Implement debounced button handling (interrupt- or polling-based) to support essential controls such as start, return-to-origin, emergency stop, and mode selection.
 - Use ADC-based potentiometer input for adjustable parameters (e.g., control gains, speed limits), ensuring sampling does not interfere with sensing or control processes.
 - implement Bluetooth-based telemetry for remote monitoring and debugging, ensuring that communication does not disrupt critical real-time execution.
 - The user interface must support safe, intuitive interaction and provide meaningful visibility into system behaviour.
- e) Real-Time Software Architecture
 - Use interrupts and DMA for efficient, low-latency operation
 - Use FreeRTOS with clearly separated tasks based on system function
 - Real-time coordination across **Sense-Think-Act** must be explicit and measurable

- f) Safety and Fault Handling (few examples)
 - Detect critical operational faults (e.g., LiDAR timeout, encoder stall, planner failure) and transition the system to a defined Fault state.
 - Implement deterministic safe-stop behaviour under fault conditions, including controlled motor shutdown.
 - Implement a latched hardware emergency stop that immediately disables PWM outputs and manual reset mechanism.
 - Implement power integrity protection (e.g., low-voltage and brown-out detection) with graceful shutdown to prevent unstable behaviour.
- g) Debugging, Validation and Data Visualisation
 - Use structured debugging to verify sensing, mapping, planning, control, and task scheduling
 - Use appropriate tools (SWV, logs, logic analyser, trace) to validate timing behaviour, interrupts, DMA, and FreeRTOS execution
 - Validate performance under realistic conditions
 - Record and visualise key parameters for tuning and benchmarking

4. Simulation Platform

Figure 2 presents a Python-based simulation platform is provided to support early-stage algorithm development and reasoning prior to hardware deployment. The simulator enables controlled design and testing of navigation strategies (e.g., exploration, global planning, and obstacle avoidance) within a maze environment, where start and exit positions may vary and the map is not pre-known.

The simulation environment supports conceptual understanding, structured experimentation, and validation of navigation logic before transferring solutions to the STM32 microcontroller platform.

Following simulation validation, students are required to re-implement and integrate the complete **Sense–Think–Act** pipeline as fully embedded firmware operating under real-time, memory, and processing constraints.

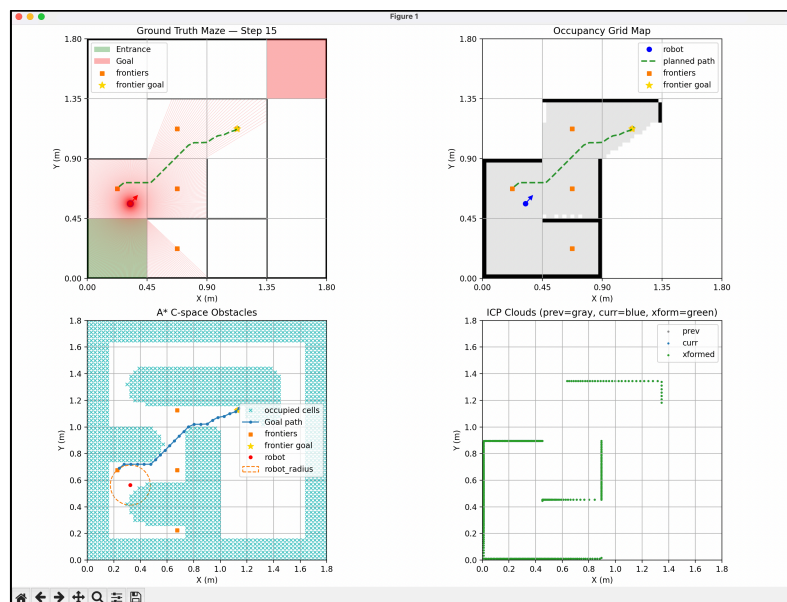


Figure 2. Python-based AMR simulation platform

This simulation-to-hardware workflow reflects professional engineering practice, where algorithms are first evaluated in controlled environments before being deployed on resource-constrained embedded systems. It supports coherent system development across algorithm design, firmware integration, and performance validation.

5. Coursework Marking Criteria

This coursework is assessed using a Hybrid Qualitative-Quantitative model within the GenAI-CDIO Authentic Assessment framework.

The final coursework mark consists of:

- Qualitative (CDIO-based evaluation) – 80%
- Quantitative (Benchmark-based performance evaluation) – 20%

The final coursework mark (0-100) is calculated as:

$$Final = 0.8 \cdot S_{CDIO} + 0.2 \cdot S_{perf}$$

The detailed descriptions of each component are provided in Sections 5.1 and 5.2.

5.1. Qualitative (CDIO-based evaluation)

The qualitative component evaluates engineering competence across the structured CDIO lifecycle. Table 1 summarises the assessment criteria and the mark distribution for each stage within the framework.

Table 1

Marking Criteria	Description	Weight (%)
1. Conceive: Critical assessment of the project challenge	Assesses the ability to interpret project requirements, define objectives, establish appropriate specifications, and frame the engineering problem in a structured and defensible manner, considering practical constraints and real-world applicability.	20%
2. Design: Higher-order Thinking	Assesses the quality of design development, including critical analysis, evaluation of alternatives, trade-off justification, and coherence of the proposed system architecture. Emphasis is placed on defensible technical reasoning rather than feature complexity.	20%
3. Implement: System Realisation and Technical Integration	Assesses the translation of design into a functional system, including hardware-software integration, structured firmware development, coordination of subsystems, and technical execution consistent with stated design decisions.	20%
4. Operate: Verification, Validation, and Performance Evaluation	Assesses the ability to systematically test and validate system behaviour, analyse performance against defined requirements, apply appropriate debugging and monitoring techniques, and provide objective evidence of system functionality under realistic conditions.	20%
5. Reflection and Self-Assessment	Assesses critical reflection across all phases of the engineering process, including evaluation of design choices, validation outcomes, limitations, and responsible use of digital tools (e.g., GenAI) with evidence of verification, refinement, and learning progression.	10%
6. Teamwork and Collaborative Learning	Assesses effective collaboration, technical communication, responsible task ownership, integration of contributions, and shared accountability in delivering a coherent engineering solution.	10%

Note that:

- Criteria 1–4 are assessed via your documentation and video presentation. These criteria measure your project outcomes in terms of technical proficiency and practical knowledge applied to real-world challenges.

- Criteria 5-6 are assessed during lab session where you will demonstrate your project process and progress, reflective activities, continuous feedback, and project quality improvement.

5.2. Quantitative (Benchmark-based performance evaluation)

The quantitative component assesses the objective performance of your system product (AMR) under standardised test conditions.

All teams will be evaluated using the maze layout shown in Figure 2, ensuring fairness, consistency, and transparency across all submissions.

The maze consists of a 5×5 grid, with each cell measuring 70 cm × 70 cm. The Start and Exit coordinates are predefined; however, the map itself is unknown and must be autonomously explored by your AMR.



Figure 2. Maze layout for AMR benchmarking

Your AMR's performance will be measured using two time-navigation metrics recorded while operating within the maze:

- Exit Time (Start → Exit) maximum 120s
- Return Time (Exit → Start) maximum 120s

Each metric is converted into a normalised performance score using clamped linear scaling:

$$P_i = \text{clamp} \left(100 \cdot \frac{T_{\text{Zero},i} - T_{\text{Team},i}}{T_{\text{Zero},i} - T_{\text{Full},i}}, 0, 100 \right)$$

where

- $T_{\text{full},i}$: Full-credit time is $\leq 30\text{s}$
- $T_{\text{zero},i}$: Zero-credit time is $\geq 120\text{s}$
- $T_{\text{Team},i}$: Team's measured performance time

The final quantitative performance score is then computed as:

$$S_{\text{perf}} = 0.5 \cdot (P_{\text{Exit}} + P_{\text{Return}})$$

For example, with $T_{\text{Team},\text{Exit}} = 80\text{s}$, $T_{\text{Team},\text{Return}} = 60\text{s}$, the normalised scores are:

$$P_{\text{Exit}} = 100 \cdot \frac{120 - 80}{120 - 30} = 33$$

$$P_{\text{Return}} = 100 \cdot \frac{120 - 60}{120 - 30} = 67$$

$$S_{\text{perf}} = 0.5 \cdot (P_{\text{Exit}} + P_{\text{Return}}) = 50$$

6. Project deliverables

Your team is required to submit the following deliverables as evidence of your engineering process, system realisation, and validated performance.

6.1. Technical Documentation Requirements (PDF, Maximum 10 Pages)

Your Technical Documentation should present clear, well-structured evidence showing how your work satisfies the marking criteria for each phase of the CDIO lifecycle (Conceive, Design, Implement, Operate) as defined in Table 1.

For each CDIO phase, your documentation must clearly address the following requirements:

- Explain the lifecycle progression by describing how the work completed in this phase builds on the previous phase and how it informs the next phase within the overall project development process.
- Provide concrete supporting evidence by including material such as problem formulation, system-architecture justification, implementation excerpts, validation data, or measured performance results that directly relate to this phase.
- Justify your engineering decisions by clearly explaining the key choices made during this phase and detailing the methods used to test, validate, or refine those decisions.
- Demonstrate the use of GenAI tools by presenting:
 - One example of a useful GenAI output that improved your reasoning
 - One example of an unhelpful or misleading output that you verified, corrected, or refined during your work

6.2. Video Submission Requirements

Two separate videos must be submitted as part of this coursework:

- Product Quality Video (Maximum 7 Minutes)
- Product Performance Benchmark Video (Separate Submission)

Each video serves a distinct assessment purpose within the hybrid qualitative–quantitative framework.

6.2.1. Product Quality Video (Maximum 7 Minutes)

This video forms part of the **Product Quality (Qualitative) Assessment** under the CDIO-based evaluation. It assesses implemented system features, integration quality, engineering reasoning, validation discipline, and real-time architecture.

This video evaluates the quality and coherence of the implemented system. It does not assess benchmark timing performance.

The video must be structured as follows:

- a) Team Introduction and Responsibilities (Approx. 1 Minute)
 - Introduce all team members
 - Briefly describe individual responsibilities and technical contributions
 - Explain how responsibilities were coordinated across the overall system architecture
- b) System Product Demonstration (Approx. 2 Minutes): Demonstrate the implemented technical requirements under realistic operating conditions. This must clearly show:
 - Functional implementation of required features
 - Coherent integration across sensing, cognition, control, and user interface components.
 - Observable cause-effect behaviour (e.g., sensor input or user interaction leading to predictable system response)
 - Stable and reliable autonomous operation
 - This section evaluates system integration and functional completeness, not timing performance or detailed validation evidence

- c) Product Evaluation (Approx. 2 Minutes): Demonstrate how system behaviour was verified, validated and refined through structured engineering practice. This may include:
 - Monitoring outputs or logged parameters
 - Real-time system state visualisation
 - Evidence of testing, refinement, and optimisation
 - This section evaluates validation discipline and evidence-based engineering reasoning
- d) Code (Approx. 2 Minutes): Explain the overall firmware structure and system organisation. Demonstrate:
 - Task organisation and scheduling strategy
 - Use of interrupts, DMA, and FreeRTOS.
 - Coordination across the Sense-Think-Act pipeline.
 - Key architectural decisions supporting real-time execution.
 - The emphasis should be on system-level clarity rather than detailed code walkthrough.

6.2.2. Product Performance Benchmark Video

This video forms part of the **Product Performance (Quantitative) Assessment (20%)** and is used solely to determine benchmark-based performance scores.

The video must:

- Record the complete Start → Exit run
- Record the complete Exit → Start return run
- Display a visible and readable timer throughout the recording

Each navigation phase is assessed independently under the 120s maximum limit (see Section 5.2 for the marking calculation).

The video may be shown at high speed (e.g., 4× or 8×), provided the timer remains clearly visible throughout. The recording must be continuous, with no cuts or edits.

The benchmark video must provide clear and traceable evidence of performance under standardised testing conditions.

6.2.3. Complete Project Code (in Zip)

You must submit a single ZIP file containing all source code, header files, configuration files, and any supporting resources required to compile and run your system on the NUCLEO-F446RE.

This submission must correspond exactly to the system demonstrated in your Product Quality Video.

You must include a **short README** file within the ZIP. The README should briefly describe:

- The overall folder structure of your project
- Any custom modules or hardware abstraction components implemented

This submission serves as supporting evidence for the functionality and behaviour demonstrated in your submitted videos.

7. Importance Dates

- Project Brief Release: **Monday (Week 3)**
- Project Start: **Monday (Week 4)**
- Project progress check: **Friday (Week 7)**
- Project live demo and interview: **TBA (Week 13)**
- Project Submission to QMPlus: **Monday (Week 14)**

----- End of Document -----